Credit Risk Classification: Technical Report

Predicting which loan applicants are likely to default, using the German Credit dataset

Author: Pranali Rokade | Date: July 2026 | Code: Credit Risk ML Pipeline

1. Executive Summary

The business problem. A bank that lends money has to decide, for every applicant, whether they will pay it back. Two mistakes are possible, and they are not equally painful. Turning away a good customer costs the bank a missed opportunity. Approving a customer who later defaults costs the bank the loan itself. Following the cost assumption in the course brief, I treated the second mistake as roughly five times more expensive than the first.

What I did. I took the German Credit dataset (1,000 past applicants, 9 pieces of information about each, and a label saying whether the loan turned out good or bad), explored it, cleaned it, and trained four kinds of model: logistic regression, a decision tree, a random forest and XGBoost. I compared them on 200 applicants the models had never seen, tried to improve the best one with hyperparameter tuning, and logged every experiment in MLflow.

What I found.

- Default XGBoost was the best model. On the 200 unseen applicants it was right 86% of the time and caught 65 of the 79 genuinely risky applicants (82.3% recall). It let 14 risky applicants slip through, the fewest of any model.
- The strongest warning signs were the applicant's job category, savings account level and checking account level. Age and sex barely mattered.
- Tuning the model did not help. The tuned version looked better during cross-validation but was no better on the test set (and missed 4 more risky applicants). I report that as it is.
- I made one methodology mistake along the way (scaling the data before splitting it), caught it, and fixed it. Section 4 explains what happened.

Recommendation. Use default XGBoost as the model that supports, not replaces, human credit officers. It should flag applications for review, and borderline cases should always go to a person. Before any real use it needs fairness checks, monitoring and more data than this small sample.

2. Data Analysis

2.1 The data

The dataset has 1,000 rows. Each row is a past loan applicant described by age, sex, job category (0 to 3), housing situation, savings account level, checking account level, credit amount, loan duration in months, and the purpose of the loan. The label is whether the credit risk was good (0) or bad (1).

2.2 Data quality

- **Duplicates:** none.

- **Missing values:** savings_account was missing for 85 applicants and checking_account for 95. Every other column was complete.
- **Class balance:** 60.4% good and 39.6% bad. This matters a lot. A lazy model that said "good" for everyone would be right 60.4% of the time while approving every bad loan. That is why I judged models on precision, recall and F1 for the bad class and not on accuracy alone.

2.3 The missing values were not random

My first instinct was to fill the gaps with the most common value. Before doing that, I tested a different idea: maybe a blank savings account means the person simply has no savings account.

The data supported this for savings. Among applicants with a blank savings account, 58.8% turned out to be bad credit. Among applicants who had a savings value, only 37.8% did. That is a big gap, so the blank itself carries information. Filling it with the most common value would have erased that signal.

For checking accounts the gap was tiny (42.1% bad when blank, against 39.3% otherwise), so there was no strong reason to treat those blanks as special. Only 6 applicants were missing both columns, so the two gaps are mostly unrelated.

Decision: I filled both blanks with an explicit category called "none", and kept a separate flag column for the savings blanks.

2.4 What predicts risk

Numeric features. Credit amount and loan duration were clearly higher for bad-credit applicants than for good ones. Age looked nearly identical between the two groups, so it is a weak predictor on its own. None of the three numeric columns was strongly skewed, so no log transform was needed.

Categorical features. The share of applicants who turned out bad, by group:

Feature	What the data showed
Savings account	little 58.7%, rich 28.2%, moderate 18.5%, quite rich 18.1%. Very strong signal.
Checking account	little 49.7%, moderate 27.2%, rich 24.4%. Strong signal.
Purpose	business 50.5% and domestic appliances 47.8% at the top, furniture 33.1% at the bottom. Moderate spread.
Sex	male 41.7%, female 37.3%. Mild difference.
Housing	rent 40.1%, own 39.6%, free 38.2%. Almost flat, so weak.

The rule of thumb I used: if the bad rate varies a lot between the groups of a feature, the feature is informative. If the rates are all close together, it is not.

2.5 Feature engineering

- **Ordinal encoding** for savings and checking accounts, because their levels have a genuine order (none < little < moderate < rich < quite rich). They became 0, 1, 2, 3, 4.

- **One-hot encoding** for sex, housing and purpose, because those categories have no natural order. Numbering them would have told the model that, say, education is "three times" car, which is meaningless.
- **Standardization** of age, credit amount and duration, so that a column measured in thousands does not overshadow a column measured in tens.
- **Stratified 80/20 split** into 800 training and 200 test rows, so both halves keep the same 60/40 good-to-bad balance.

3. Methodology

3.1 Why these four models

I started with the simplest model and moved toward more powerful ones, so I could see what extra complexity actually buys.

- **Logistic regression:** a simple, interpretable baseline. It gives each feature a weight and turns the weighted sum into a probability of being a bad risk.
- **Decision tree:** a chain of yes-or-no questions. It can capture combinations of features that logistic regression misses.
- **Random forest:** many trees voting together, which reduces the memorization problem of a single tree.
- **XGBoost:** many trees built one after another, each one concentrating on the mistakes of the previous ones.

All four were trained on the same 800 applicants and tested on the same 200, so the comparison is fair. Every random choice used a fixed seed (42), so the results can be reproduced.

3.2 How I judged the models

Accuracy is easy to understand but misleading with imbalanced classes. For the bad class I looked at:

- **Precision:** of the applicants the model flagged as risky, how many really were.
- **Recall:** of the applicants who really were risky, how many the model caught. This is the metric closest to the bank's main worry.
- **F1:** a single number that blends precision and recall.
- **ROC AUC:** how well the model ranks risky applicants above safe ones across every possible cutoff.
- **The confusion matrix:** the raw count of correct calls and each type of mistake.

3.3 Tuning strategy

I ran a grid search on XGBoost over the number of trees (50, 100, 200), tree depth (3, 4, 5, 6) and learning rate (0.01, 0.1, 0.2). That is 36 combinations, each scored with 5-fold cross-validation on the training data, optimizing for recall. Cross-validation splits the training data into five parts and rotates which part is held out, which gives a steadier estimate than a single split.

I also tested lowering the decision cutoff from 0.5 down to 0.25, to trade a few extra false alarms for fewer missed defaulters.

3.4 Experiment tracking

All six models (logistic regression, both decision trees, random forest, default XGBoost and tuned XGBoost) were logged to an MLflow experiment called credit-risk-classification, with their metrics and the number of each type of mistake, so any run can be compared or reproduced later.

4. A Mistake I Caught and Fixed: Data Leakage

This section is the part of the project I learned the most from.

What I did wrong. In my first version of the pipeline, I standardized the three numeric columns using all 1,000 rows, and only afterwards split the data into training and test sets. To standardize a column, the scaler works out its average and spread. Because I did that on all 1,000 rows, those averages included the 200 test applicants. A small amount of information about the test set had leaked into the way the training data was prepared.

An everyday way to think about it: it is like studying for an exam after being told the class average on that exam. You never saw the questions, but you got a small hint you would never have in real life. That makes the test less honest.

How I noticed. I reviewed the order of my steps against the project brief, which asks for a proper pipeline, and saw that scaling came before the split.

How I fixed it. The correct order is split first, then learn the averages from the training set only, then apply those same numbers to both sets. In code, that means calling fit_transform on the training data and plain transform on the test data. The test set is treated like future applicants: I may use the training numbers on them, but never learn from them. I removed the old scaling step, added the corrected one after the split, restarted the notebook and re-ran everything from the top.

What changed. Almost nothing, which is what I expected. Three of my four models are tree-based and do not care about feature scale. Only logistic regression could have been affected, and its training accuracy moved from 80.1% to 80.0% while its test results stayed identical. The reported results in this document are all from the corrected version.

Why I am keeping this in the report. The effect was tiny, but the habit matters: any step that learns something from the data (scaling, filling blanks, encoding) must be fitted on the training set alone, so the test set stays a fair stand-in for applicants the model has never seen. A natural next improvement is to wrap preprocessing and the model into a single scikit-learn Pipeline, which enforces this automatically.

5. Results

5.1 Model comparison

All numbers are on the 200 unseen test applicants (79 genuinely risky, 121 genuinely good).

Model	Train acc.	Test acc.	Precision	Recall	F1	ROC AUC	Missed risky	False alarms

Model	Train acc.	Test acc.	Precision	Recall	F1	ROC AUC	Missed risky	False alarms
Logistic regression	0.800	0.765	0.722	0.658	0.689	0.845	27	20
Decision tree (no limit)	1.000	0.840	0.813	0.772	0.792	0.828	18	14
Decision tree (depth 4)	0.839	0.800	0.855	0.595	0.701	0.880	32	8
Random forest	1.000	0.830	0.817	0.734	0.773	0.930	21	13
XGBoost (default)	1.000	**0.860**	0.823	**0.823**	0.823	0.935	**14**	14
XGBoost (tuned)	1.000	0.855	0.847	0.772	0.808	0.937	18	11

"Missed risky" means a truly bad applicant the model called good. "False alarms" means a truly good applicant the model called bad.

5.2 Reading the confusion matrix for the final model

For default XGBoost the matrix was 107, 14 on the top row and 14, 65 on the bottom row. Rows are the truth and columns are the model's call:

- **107** good applicants correctly approved.
- **14** good applicants wrongly flagged as risky (false alarms).
- **14** risky applicants wrongly approved. These are the costly mistakes.
- **65** risky applicants correctly caught.

5.3 What the comparison taught me

Overfitting is visible in the gap. The unrestricted decision tree scored 100% on its training data but 84% on the test set. A 16-point gap means it memorized the 800 training applicants, quirks included. Limiting its depth to 4 closed the gap (83.9% train, 80% test), but recall collapsed to 59.5% and it missed 32 risky applicants, worse than logistic regression. Fixing overfitting did not automatically give a better model for the bank's goal.

A 100% training score is not always a problem. Random forest and XGBoost also hit 100% on training data, but they combine many trees, so they held up on unseen applicants. The gap between training and test is the thing to watch, together with how the model performs on the unseen set.

A cost-based view. Using the brief's assumption that a missed defaulter costs five times a false alarm, a rough cost score (missed risky x 5 + false alarms x 1) gives: logistic regression 155, unrestricted tree 104, depth-4 tree 168, random forest 118, default XGBoost 84, tuned XGBoost 101. Default XGBoost is the cheapest.

ROC curves. The random forest and both XGBoost models sit close together near the top-left corner of the ROC chart, with AUCs of 0.930 to 0.937. The differences between them are too small to rank confidently on 200 test rows. The unrestricted tree has the weakest curve because a fully grown tree outputs mostly hard 0 or 1 scores, with nothing in between.

5.4 What the model relies on

The most important features for XGBoost were job (0.18), savings account (0.17) and checking account (0.13), followed by the domestic appliances purpose (0.08), credit amount (0.07) and duration (0.06). Age (0.05) and sex (0.01) ranked low. This matches what I saw by eye in the exploratory analysis, which is reassuring: the model learned the patterns that were really in the data.

5.5 Who slips through

I looked at the 14 risky applicants the final model approved. On average they had moderate savings (about 2.0 on the 0 to 4 scale) and checking levels around "little to moderate", so they did not show the classic warning signs the model relies on most. Their credit amount and duration were only mildly above average. These are borderline cases, and no model will catch every one of them. Some risk hides behind ordinary-looking numbers.

5.6 Tuning did not help

The grid search picked a learning rate of 0.2, a depth of 5 and 200 trees, and reported a cross-validated recall of 84.2%. On the test set, however, that model had a recall of 77.2% against 82.3% for the default. These two numbers are not directly comparable: one is an average over folds of the training data, the other comes from a single set of 200 people, of whom only 79 are risky. Each person missed moves recall by about 1.3 percentage points, so a difference of 4 people is partly noise. The honest conclusion is that tuning gave no clear improvement, probably because the defaults were already close to a good setting for a dataset this small.

5.7 Changing the cutoff

Lowering the cutoff from 0.5 to 0.25 caught 2 more risky applicants (14 missed down to 12) at the price of 3 more false alarms (14 up to 17). The effect is small because XGBoost is very confident in its probabilities, so few applicants sit in the zone between 0.25 and 0.5. Because I looked at the test set while choosing the cutoff, this result is slightly flattering, and I kept the default 0.5.

6. Business Recommendations

Recommended model: default XGBoost. It had the highest accuracy (86%), the highest recall for risky applicants (82.3%), the fewest missed defaulters (14 of 79), a balanced precision, and the lowest cost score. It ranks applicants about as well as the tuned model and the random forest, but it needs no hand-picked settings.

How I would use it.

- As decision support. The model proposes, a credit officer decides. Applicants with a high risk probability get a review, not an automatic rejection.
- Keep a human review for borderline probabilities, since those are exactly the cases the model finds hardest.
- Keep the explanation available. Savings, checking account and job are the main drivers, which helps explain a decision to a customer or a regulator.
- Treat the cutoff as a business decision. If the bank decides a missed defaulter is more costly than 5 false alarms, the cutoff can be lowered, with the trade-off shown above.

7. Deployment Considerations

This project did not deploy the model, but here is what a real deployment would need.

Infrastructure. Save the preprocessing and the model as one pipeline object, so applications are transformed exactly the way training data was. Serve it behind a small API (for example a REST service) that takes an application and returns a risk probability. A dataset this size needs very modest hardware.

Monitoring.

- Data drift: track whether incoming applicants still look like the training data (average loan size, mix of savings levels, and so on). Economic changes can shift this quickly.
- Performance: once real repayment outcomes arrive, recompute recall, precision and the number of missed defaulters on a regular schedule. Outcomes arrive late, so this is slow feedback.
- Input quality: watch for a rising share of blank savings or checking values, since that blank carries meaning in this model.

Retraining and versioning. Retrain on a schedule or when drift is detected, and keep each model version, its settings and its metrics in an experiment tracker like MLflow, so a decision can always be traced back to the model that made it.

Risks and challenges.

- **Small sample.** 1,000 applicants and 79 risky cases in the test set means every number here has a wide margin of error.
- **Fairness and regulation.** Sex is in the data and showed a mild difference in default rates. In real lending, using such attributes can be restricted or illegal, and proxies for them can slip in through other features. A real deployment would need fairness testing across groups and a legal review before launch.
- **Selection bias.** The data only contains applicants who were approved and whose outcome is known. The model has never seen the ones who were rejected.
- **Over-trust.** An 86% accurate model still makes mistakes on one applicant in seven. Those who are denied need a way to appeal.

8. Limitations

I want to be upfront about what this project cannot claim.

- I chose the final model by looking at test-set scores, so those scores are slightly optimistic. A cleaner design would pick the model using a separate validation set and touch the test set once at the very end.
- The test set is small. Differences of one or two applicants between models are within noise, and I have not run repeated cross-validation to put error bars on the results.
- The 5-to-1 cost ratio is an assumption from the brief, not a real bank's number.
- The tuning search was small, and only recall was optimized.

9. Conclusion and Future Work

The main outcome is a working, honest pipeline: clean the data, test an idea about the missing values before acting on it, compare four models fairly, and judge them on the mistake that matters most to a lender. XGBoost came out on top, and the features it relied on matched what the exploratory analysis had already shown.

The most valuable lessons were not about accuracy. A perfect training score is a warning sign for a single tree. A tuned model is not automatically a better one. A small leakage mistake is worth finding and owning.

Future work.

- Wrap preprocessing and the model in a scikit-learn Pipeline, so leakage cannot happen by construction.
- Select the model and the cutoff on a validation set, and use repeated cross-validation to get confidence intervals on the metrics.
- Try class weights or resampling, and probability calibration, to make the risk percentages more trustworthy.
- Add model explanations such as SHAP values, to explain individual decisions.
- Run a fairness analysis across groups and test a version of the model without sex.
- Add the model to a small API with a monitoring dashboard.

Appendix

- GitHub repository: https://github.com/pranalirokade/credit-risk-ml-pipeline
- Notebook: credit_risk_pipeline.ipynb (in the Notebooks folder)
- MLflow experiment: credit-risk-classification (6 runs)
- Dataset: German Credit data, course-provided version, originally from the UCI Machine Learning Repository
- Charts (Visualizations folder): ROC curves, XGBoost confusion matrix, XGBoost feature importance